Fuzzy matching para el TP4

Oct 17, 2026 · @BELEN

1. Qué problema resuelve

El *fuzzy matching* asigna a cada texto mal escrito su versión correcta cuando no coinciden letra por letra. Es una herramienta de **sugerencia**, no de decisión: propone candidatos con un puntaje, y ustedes deciden cuáles aceptar.

En economía aparece todo el tiempo:

- **Unificar categorías** dentro de una base: `BCO NACION`, `Banco de la Nación` y `BANCO NACION ARG.` son la misma entidad.
- **Unir dos bases sin un identificador común**: empresas de un registro con firmas de una encuesta, municipios escritos distinto en dos fuentes, personas en censos de años diferentes.

Un `merge` de pandas exige igualdad exacta: una tilde o un espacio de más dejan la fila afuera, sin aviso. El *fuzzy matching* rescata esas filas, pero también puede **inventar uniones que no existen**. Todo este documento trata de aprovechar lo primero sin caer en lo segundo.

En el TP4 lo usan en el ejercicio 7, para unificar nombres de barrios. Acá van la teoría y ejemplos con otros datos.

2. La teoría: cuántas ediciones separan dos textos

Dos textos se parecen más cuantas menos operaciones hacen falta para transformar uno en el otro. Esa cantidad mínima se llama **distancia de edición**, y hay dos versiones que importan acá:

Distancia	Operaciones permitidas	Costo de cambiar una letra	`CASA` → `CAZA`
Levenshtein	insertar, borrar, sustituir	1 (una sustitución)	1
Indel	solo insertar y borrar	2 (borrar y volver a insertar)	2

La función `fuzz.ratio` de RapidFuzz usa **Indel** y la convierte en un puntaje de 0 a 100, dividiendo por el largo total de los dos textos:

```
\text{ratio}(a,b) = 100 \left(1 - \frac{\text{Indel}(a,b)}{|a| + |b|}\right)
= 100 \cdot \frac{2 \cdot \text{LCS}(a,b)}{|a| + |b|}
```

LCS es la subsecuencia común más larga: las letras que aparecen en ambos textos en el mismo orden, aunque no estén pegadas. La segunda forma es la más intuitiva: **el puntaje es la proporción de letras que los dos textos comparten en orden.**

Ejemplo a mano. `CASA` y `CAZA` comparten `C`, `A` y `A` en orden, así que LCS = 3. El largo total es 4 + 4 = 8. Entonces ratio = 100 × 2 × 3 / 8 = **75**.

Tres consecuencias que conviene tener presentes:

1. **Es letra por letra, y las mayúsculas y tildes cuentan.** `Termica` contra `termica` da 85,7, igual que `TÉRMICA` contra `TERMICA`: para el algoritmo, `T` y `t` son tan distintas como `T` y `X`.
2. **El orden importa.** `SAN NICOLAS TERMICA` contra `TERMICA SAN NICOLAS` da 57,9, aunque tengan las mismas palabras.
3. **No sabe qué significan las letras.** Un dígito distinto pesa lo mismo que una letra distinta, aunque `VILLA 31` y `VILLA 21` sean lugares diferentes (87,5 puntos).

Para calcular la distancia se usa **programación dinámica**: una tabla donde la celda (i, j) guarda la distancia entre los primeros i caracteres de un texto y los primeros j del otro, y cada celda se llena a partir de sus vecinas. El costo crece con el producto de los largos; RapidFuzz lo acelera con operaciones sobre bits, pero la idea es esa.

Hay otras medidas, también en RapidFuzz (`rapidfuzz.distance`): **Damerau-Levenshtein** cuenta como una sola edición el cambio de dos letras vecinas (`TREMICA`), y **Jaro-Winkler** premia que el comienzo coincida, por eso es la más usada para apellidos en *record linkage*.

3. La librería: RapidFuzz y sus puntajes

Usamos RapidFuzz: es rápida (está escrita en C++), tiene licencia MIT y está mantenida. En materiales viejos van a ver `fuzzywuzzy`: fue archivada en 2024 y renombrada `thefuzz`. La interfaz es casi igual, con una diferencia importante que vemos en la sección 4.

```
# en la terminal, con el entorno del curso activado
pip install rapidfuzz
# y en requirements.txt
rapidfuzz
```

Todos los puntajes de `fuzz` devuelven un número de 0 a 100, y todos usan por dentro el `ratio` de la sección 2. Lo que cambia es **qué** comparan:

Función	Qué hace antes de comparar	Ejemplo	Puntaje	Cuándo sirve
`fuzz.ratio`	nada: compara los textos enteros	`TERMCA` / `TERMICA`	92,3	errores de tipeo en textos de largo parecido
`fuzz.partial_ratio`	busca el mejor tramo del texto largo, del largo del corto	`TERMICA` / `CENTRAL TERMICA SAN NICOLAS`	100	el texto corto está **dentro** del largo
`fuzz.token_sort_ratio`	separa en palabras, las ordena alfabéticamente y une	`SAN NICOLAS TERMICA` / `TERMICA SAN NICOLAS`	100	mismas palabras en otro orden
`fuzz.token_set_ratio`	compara las palabras en común con las que sobran en cada lado	`BANCO NACION` / `BANCO DE LA NACION ARGENTINA`	100	un texto agrega palabras al otro
`fuzz.WRatio`	calcula varios de los anteriores y se queda con el mejor, ponderado según cuánto difieren los largos			puntaje por defecto de `process`

La misma propiedad que hace útil a cada función la vuelve peligrosa:

- `partial_ratio` y `token_set_ratio` dan **100** a `BANCO NACION` contra `BANCO NACION SEGUROS`, que es otra empresa. Si todas las palabras de un texto están en el otro, el puntaje es máximo.
- `token_sort_ratio` no distingue `JUAN PEREZ` de `PEREZ JUAN`, lo que a veces es justo lo que buscan y a veces no.

- `WRatio` es cómodo pero opaco: no sabrán cuál de las comparaciones decidió el puntaje. Si van a justificar un umbral, conviene elegir el puntaje a propósito.

4. Buscar el mejor candidato: `process`

El módulo `process` compara un texto contra una lista de opciones válidas y devuelve las mejores. Los valores por defecto, según la documentación:

Función	Devuelve	Puntaje por defecto	Preprocesa por defecto
`process.extractOne(texto, opciones)`	una tupla `(opción, puntaje, posición)`, o `None` si nada supera `score_cutoff`	`WRatio`	**no**
`process.extract(texto, opciones, limit=5)`	una lista de esas tuplas, de mejor a peor	`WRatio`	**no**
`process.cdist(textos, opciones)`	una matriz con todos los puntajes	`ratio`	**no**

La diferencia con `fuzzywuzzy`: RapidFuzz no pasa a minúsculas ni limpia signos salvo que se lo pidan con `processor=utils.default_process`. Esa función pasa a minúsculas, reemplaza todo lo que no es letra o número por un espacio y recorta los extremos. **No saca tildes**: eso lo hace su propia función de normalización.

```
from rapidfuzz import process, fuzz, utils

validas = ["HIDRAULICA", "TERMICA", "EOLICA", "SOLAR", "NUCLEAR"]

process.extractOne("Termca", validas,
                   scorer=fuzz.ratio, processor=utils.default_process)
# ('TERMICA', 92.3..., 1)

process.extract("Termca", validas, scorer=fuzz.ratio,
                processor=utils.default_process, limit=3)
# lista con los tres mejores: sirve para ver qué tan cerca quedó el segundo
```

Y la trampa central, si la opción correcta **no está** en la lista:

```
sin_nuclear = ["HIDRAULICA", "TERMICA", "EOLICA", "SOLAR"]

process.extractOne("NUCLEAR", sin_nuclear, scorer=fuzz.ratio)
# ('SOLAR', 50.0, 3)  <- devuelve la menos mala, sin aviso

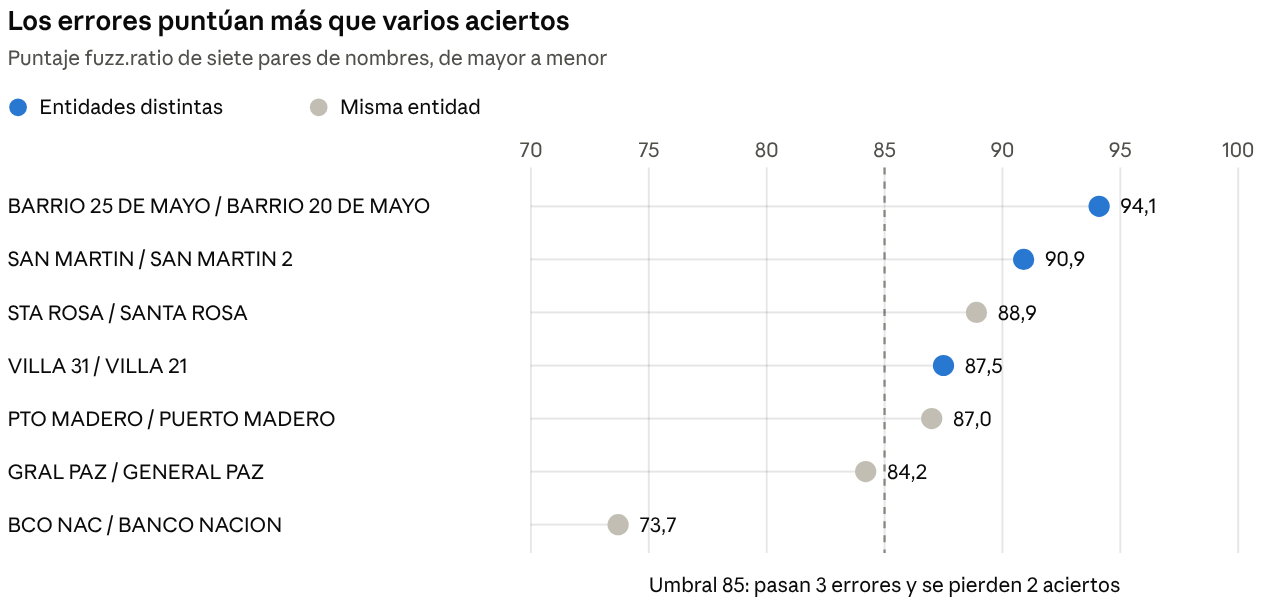
process.extractOne("NUCLEAR", sin_nuclear, scorer=fuzz.ratio,
score_cutoff=85)
# None  <- con umbral, reconoce que no hay candidato
```

Regla práctica: **apliquen `extractOne` sobre los valores distintos** (`df["col"].unique()`), no fila por fila con `apply`. Con miles de filas y unos cientos de nombres distintos, es muchísimo más rápido y, sobre todo, les deja una tabla chica que pueden leer entera.

5. Las cuatro trampas

El error más caro del *fuzzy matching* es silencioso: una unión equivocada no da error, da un número.

1. **Siempre devuelve algo.** Sin `score_cutoff`, `extractOne` asigna la opción menos mala aunque la correcta no exista (sección 4).
2. **Los números pesan como una letra.** `BARRIO 25 DE MAYO` contra `BARRIO 20 DE MAYO` da 94,1, y son barrios distintos. Para el algoritmo cambió un carácter; para ustedes cambió el lugar.
3. **Las abreviaturas puntúan bajo.** `BCO NAC` contra `BANCO NACION` da 73,7, aunque es la misma entidad: abreviar borra letras, y cada letra borrada resta.
4. **Por 2 y 3, ningún umbral separa aciertos de errores.** En el gráfico de abajo, los errores quedan **por encima** de varios aciertos. Si suben el umbral para frenar los errores pierden los aciertos con abreviaturas; si lo bajan, entran los errores.



fuzz.ratio calculado para este documento · 7 pares de ejemplo

Con cualquier umbral entre 74 y 94 pasa al menos un error o se pierde al menos un acierto.

La salida no es buscar el umbral perfecto, porque no existe. Es combinar el puntaje con **reglas** que expresan lo que ustedes saben del dominio: por ejemplo, «si los dos nombres tienen números, tienen que ser los mismos números». La regla no la puede inventar el algoritmo, y tampoco la IA: escribe código de fuzzy matching en segundos, pero no sabe qué es un error en sus datos. Eso sale de mirar los datos.

6. El flujo de trabajo que recomendamos

El *fuzzy matching* es un paso dentro de un proceso, no el proceso entero. El orden importa:

1. **Normalizar** los dos lados igual: mayúsculas, sin tildes, sin signos, espacios simples. Mucho de lo que parecía *fuzzy* se resuelve acá.
2. **Resolver primero las coincidencias exactas** (después de normalizar) y dejar el *fuzzy* solo para el resto.
3. **Definir la lista de referencia**: de una fuente oficial o, si no hay, de los valores más frecuentes revisados a mano. Si el destino correcto no está en la lista, ningún algoritmo lo encuentra.
4. **Buscar candidatos sobre los valores distintos**, con un puntaje elegido a propósito, y guardar también el segundo mejor.

5. **Aplicar reglas de dominio y un umbral.** Lo que no las cumple queda sin asignar; mejor un faltante visible que un error escondido.
6. **Armar una tabla auditable** y leerla, empezando por los puntajes más bajos que aceptaron.
7. **Verificar y reportar**: cuántos valores y filas quedaron asignados, cuántos no, y qué nombres originales cayeron en cada destino. Si la auditoría encuentra errores, vuelvan al paso 5.

Un esqueleto para los pasos 4 a 6, con un ejemplo de empresas. La función `regla_ok` la escriben ustedes según sus datos:

```
import pandas as pd
from rapidfuzz import process, fuzz

def regla_ok(original, candidato):
    # ejemplo: si hay números, tienen que coincidir
    ...

filas = []
for nombre in sin_match_exacto:          # valores distintos, ya normalizados
    top = process.extract(nombre, referencia, scorer=fuzz.ratio, limit=2)
    (cand, punt, _), (cand2, punt2, _) = top
    acepta = punt >= UMBRAL and regla_ok(nombre, cand)
    filas.append({"original": nombre, "candidato": cand, "puntaje": punt,
                  "segundo": cand2, "puntaje_2": punt2,
                  "asignado": cand if acepta else None})

auditoria = pd.DataFrame(filas).sort_values("puntaje")
```

Dos lecturas de esa tabla detectan casi todos los errores:

- **Por destino**: `auditoria.groupby("asignado")["original"].apply(list)`. Si a un destino llegan nombres que no deberían estar juntos, ahí lo ven.
- **Por margen**: si `puntaje` y `puntaje_2` están muy cerca, el algoritmo dudó entre dos opciones. Revisen esas filas a mano.

7. Práctica opcional

Esta sección es opcional y no se evalúa. La sumamos porque este tipo de práctica es muy útil para quienes trabajen con bases que tengan estos desafíos, por ejemplo en su tesis.

En cada ejercicio, **escriban su predicción antes de ejecutar** y anoten si acertaron. Predecir y equivocarse enseña más que ejecutar y leer.

1. **A mano.** Calculen `fuzz.ratio("MARTA", "MARTHA")` con la fórmula de la sección 2. Después compruébenlo en Python.
2. **Ordenar puntajes.** Para `JUAN PEREZ` y `PEREZ JUAN CARLOS`, ordenen de mayor a menor los puntajes de `ratio`, `partial_ratio`, `token_sort_ratio` y `token_set_ratio`, y expliquen en una línea cada posición. Ejecuten y comparen.
3. **Romperlo a propósito.** Armen una lista de referencia de cinco nombres y un texto para el que `extractOne` devuelva una opción **equivocada con más de 90 puntos**. Después escriban la regla más simple que lo bloquee sin bloquear los aciertos.
4. **El acierto que se pierde.** Encuentren un par que sea la misma entidad y puntúe menos de 75. ¿Qué paso de normalización lo subiría?
5. **Auditar.** En la práctica de la clase (centrales y fuentes de generación), agrupen la tabla por destino y respondan: ¿hay algún destino que reciba nombres que no deberían estar juntos?

- [ ] Ejercicios 1 y 2 hechos sin IA
- [ ] Ejercicio 3: lista, texto y regla documentados
- [ ] Ejercicio 5: tabla por destino revisada

8. Para la tesis: desafíos y errores de medición

Cuando unan bases por nombre, cada decisión tiene dos errores posibles, y su costo no es simétrico:

- **Falso positivo**: unir registros que no son la misma entidad. Mete ruido en las variables y suele atenuar los coeficientes hacia cero.
- **Falso negativo**: no unir registros que sí lo son. Achica la muestra y, si los que no se unen son distintos (por ejemplo, nombres más largos o más abreviados), introduce sesgo de selección.

Subir el umbral cambia falsos positivos por falsos negativos. Por eso en la tesis no alcanza con reportar «usé *fuzzy matching*»: reporten la tasa de unión, el umbral y las reglas, una auditoría manual de una muestra y si los resultados cambian con otro umbral.

La literatura llama a esto *record linkage*. Tres lecturas para quien lo necesite:

- Fellegi y Sunter (1969), «A Theory for Record Linkage», *Journal of the American Statistical Association*: el modelo probabilístico clásico.

- Enamorado, Fifield e Imai (2019), «Using a Probabilistic Model to Assist Merging of Large-Scale Administrative Records», *American Political Science Review*: el paquete `fastLink`.
- Abramitzky, Boustan, Eriksson, Feigenbaum y Pérez (2021), «Automated Linking of Historical Data», *Journal of Economic Literature*: compara métodos y cómo cambian los resultados económicos.

Fuentes

- RapidFuzz, repositorio
- RapidFuzz, documentación de `process`
- fuzzywuzzy, repositorio archivado

Los puntajes de este documento se calcularon con la fórmula de `fuzz.ratio`, `token_sort_ratio` y `token_set_ratio`; conviene volver a ejecutarlos con la versión de RapidFuzz del `requirements.txt` del curso.